DAY 6 – DATA MANIPULATION USING DPLYR

1. Introduction to dplyr

`dplyr` is an R package used for data manipulation and transformation.

It helps Data Analysts to:

1. Select specific columns.
2. Filter specific records.
3. Create new columns.
4. Sort data.
5. Calculate totals and averages.
6. Perform group-wise analysis.

Example: A company has employee data. We need to find employees earning more than ₹50,000, calculate annual salaries and find department-wise average salaries.

2. Installing and Loading dplyr

Step 1: Install the package.

```
install.packages("dplyr")
```

Step 2: Load the package.

```
library(dplyr)
```

3. Create Employee Dataset

We will use the following dataset for all practical examples.

```
employees <- data.frame(
  Employee_ID = c(101, 102, 103, 104, 105, 106),
  Name = c("Amit", "Rahul", "Priya", "Neha", "Vikas", "Sneha"),
  Department = c("Sales", "IT", "Sales", "HR", "IT", "Sales"),
  Salary = c(35000, 70000, 55000, 45000, 85000, 65000),
  Experience = c(2, 5, 3, 2, 7, 4)
)

employees
```

4. Understanding the Pipe Operator (%>%)

The pipe operator `%>%` passes the output of one operation into the next operation.

It makes code easier to read and understand.

Without pipe:

```
arrange(
  filter(employees, Salary > 50000),
  desc(Salary)
)
```

With pipe:

```
employees %>%
  filter(Salary > 50000) %>%
  arrange(desc(Salary))
```

The second method is easier to understand.

5. SELECT() – Selecting Columns

The `select()` function is used to select specific columns from a dataset.

Syntax:

```
select(data, column_name)
```

Example: Select only Name and Salary.

```
employees %>%
  select(Name, Salary)
```

Select multiple columns:

```
employees %>%
  select(Name, Department, Salary)
```

Select all columns except Employee_ID:

```
employees %>%
  select(-Employee_ID)
```

Business Use: Display only the information required for a report.

6. FILTER() – Filtering Rows

The `filter()` function is used to extract records based on conditions.

Syntax:

```
filter(data, condition)
```

Example 1: Employees earning more than 50,000.

```
employees %>%
  filter(Salary > 50000)
```

Example 2: Employees working in Sales.

```
employees %>%
  filter(Department == "Sales")
```

Example 3: Employees earning more than 50,000 and working in Sales.

```
employees %>%
  filter(
    Salary > 50000,
    Department == "Sales"
  )
```

Example 4: Employees working in Sales OR IT.

```
employees %>%
  filter(
    Department == "Sales" |
    Department == "IT"
  )
```

Example 5: Employees with experience greater than or equal to 4 years.

```
employees %>%
  filter(Experience >= 4)
```

Important Operators:

Operator	Meaning
`>`	Greater than
`<`	Less than

`>=`	Greater than or equal
`<=`	Less than or equal
`==`	Equal to
`!=`	Not equal to
`&`	AND
`	`

7. MUTATE() – Creating New Columns

The `mutate()` function is used to create new columns or modify existing columns.

Example 1: Calculate annual salary.

```
employees <- employees %>%
  mutate(
    Annual_Salary = Salary * 12
  )
```

Example 2: Calculate 10% bonus.

```
employees <- employees %>%
  mutate(
    Bonus = Salary * 0.10
  )
```

Example 3: Calculate salary after bonus.

```
employees <- employees %>%
  mutate(
    Total_Salary = Salary + Bonus
  )
```

Example 4: Create employee category.

```
employees <- employees %>%
  mutate(
    Salary_Category = ifelse(
      Salary >= 60000,
      "High Salary",
      "Normal Salary"
    )
  )
```

Business Use: Create calculated columns for MIS and salary reports.

8. ARRANGE() – Sorting Data

The `arrange()` function is used to sort records.

Example 1: Sort salary in ascending order.

```
employees %>%
  arrange(Salary)
```

Example 2: Sort salary in descending order.

```
employees %>%
  arrange(desc(Salary))
```

Example 3: Sort by experience.

```
employees %>%
  arrange(desc(Experience))
```

Example 4: Sort by department and salary.

```
employees %>%
  arrange(Department, desc(Salary))
```

Business Use: Identify highest-paid employees.

9. SUMMARISE() – Calculating Summary

The `summarise()` function is used to calculate summary values.

Example 1: Calculate average salary.

```
employees %>%
  summarise(
```

```
    Average_Salary = mean(Salary)
  )
```

Example 2: Calculate total salary.

```
employees %>%
  summarise(
    Total_Salary = sum(Salary)
  )
```

Example 3: Calculate multiple summaries.

```
employees %>%
  summarise(
    Total_Salary = sum(Salary),
    Average_Salary = mean(Salary),
    Maximum_Salary = max(Salary),
    Minimum_Salary = min(Salary),
    Employee_Count = n()
  )
```

Important Summary Functions:

Function	Purpose
`sum()`	Total
`mean()`	Average
`max()`	Maximum
`min()`	Minimum
`n()`	Number of records
`median()`	Middle value

10. GROUP_BY() – Group-wise Analysis

The `group_by()` function is used to divide data into groups based on a column.

It is commonly used with `summarise()`.

Example 1: Department-wise average salary.

```
employees %>%
  group_by(Department) %>%
  summarise(
    Average_Salary = mean(Salary)
  )
```

Example 2: Department-wise total salary.

```
employees %>%
  group_by(Department) %>%
  summarise(
    Total_Salary = sum(Salary)
  )
```

Example 3: Department-wise employee count.

```
employees %>%
  group_by(Department) %>%
  summarise(
    Total_Employees = n()
  )
```

Example 4: Multiple calculations department-wise.

```
employees %>%
  group_by(Department) %>%
  summarise(
    Total_Employees = n(),
    Total_Salary = sum(Salary),
    Average_Salary = mean(Salary),
    Maximum_Salary = max(Salary)
  )
```

Business Use: Prepare department-wise salary and employee reports.

11. Combining Multiple Functions

We can combine different dplyr functions to perform complete analysis.

Example: Find Sales department employees earning more than ₹50,000 and display them in descending salary order.

```
employees %>%
  filter(
    Department == "Sales",
    Salary > 50000
  ) %>%
  select(Name, Department, Salary) %>%
  arrange(desc(Salary))
```

Example: Calculate department-wise average salary and sort highest average first.

```
employees %>%
  group_by(Department) %>%
  summarise(
    Average_Salary = mean(Salary)
  ) %>%
  arrange(desc(Average_Salary))
```

12. Complete Practical Example – Employee Analysis

Perform the following tasks using the employee dataset:

```
library(dplyr)

employees <- data.frame(
  Employee_ID = c(101, 102, 103, 104, 105, 106),
  Name = c("Amit", "Rahul", "Priya", "Neha", "Vikas", "Sneha"),
  Department = c("Sales", "IT", "Sales", "HR", "IT", "Sales"),
  Salary = c(35000, 70000, 55000, 45000, 85000, 65000),
  Experience = c(2, 5, 3, 2, 7, 4)
)
```

Task 1: Display Name and Salary columns.

```
employees %>%
  select(Name, Salary)
```

Task 2: Find employees earning above ₹50,000.

```
employees %>%
  filter(Salary > 50000)
```

Task 3: Calculate annual salary.

```
employees <- employees %>%
  mutate(Annual_Salary = Salary * 12)
```

Task 4: Calculate 10% bonus.

```
employees <- employees %>%
  mutate(Bonus = Salary * 0.10)
```

Task 5: Sort employees by highest salary.

```
employees %>%
  arrange(desc(Salary))
```

Task 6: Calculate average salary.

```
employees %>%
  summarise(
    Average_Salary = mean(Salary)
  )
```

Task 7: Calculate department-wise average salary.

```
employees %>%
  group_by(Department) %>%
  summarise(
    Average_Salary = mean(Salary)
  )
```

Task 8: Find the department with the highest total salary.

```
employees %>%
  group_by(Department) %>%
  summarise(
    Total_Salary = sum(Salary)
  ) %>%
  arrange(desc(Total_Salary))
```

13. Student Practice Questions

Use the employee dataset and solve the following:

1. Display only Name and Department.

2. Find employees earning more than ₹60,000.
3. Find employees working in the IT department.
4. Find employees having experience greater than 3 years.
5. Find employees earning less than ₹50,000.
6. Create a new column called Annual_Salary.
7. Create a Bonus column using 15% of salary.
8. Sort employees by salary in ascending order.
9. Sort employees by salary in descending order.
10. Calculate total salary of all employees.
11. Calculate average salary.
12. Calculate maximum and minimum salary.
13. Calculate department-wise employee count.
14. Calculate department-wise total salary.
15. Calculate department-wise average salary.

14. Homework Assignment – Sales Data Analysis

Create the following dataset:

```
sales <- data.frame(
  Product = c("Laptop", "Mouse", "Keyboard", "Monitor", "Laptop",
"Mouse"),
  Region = c("West", "North", "South", "West", "East", "North"),
  Quantity = c(2, 10, 5, 3, 1, 8),
  Sales = c(120000, 10000, 15000, 60000, 65000, 8000)
)
```

Perform the following tasks:

1. Display Product and Sales columns.
2. Find sales greater than ₹20,000.
3. Filter records for the West region.
4. Create a new column called Revenue_Per_Unit.
5. Sort sales from highest to lowest.
6. Calculate total sales.
7. Calculate average sales.
8. Find region-wise total sales.
9. Find product-wise total sales.

10. Identify the best-performing region.

Expected Output: A complete sales analysis report using dplyr.

15. Important Functions – Day 6

Function	Purpose
`library()`	Load package
`select()`	Select columns
`filter()`	Filter rows
`mutate()`	Create or modify columns
`arrange()`	Sort records
`desc()`	Descending order
`summarise()`	Calculate summaries
`group_by()`	Group records
`%>%`	Pipe operator
`n()`	Count records

`mean()`	Average
`sum()`	Total
`max()`	Maximum
`min()`	Minimum

16. Data Analyst Workflow

IMPORT DATA

↓

SELECT REQUIRED COLUMNS

↓

FILTER REQUIRED RECORDS

↓

CREATE NEW COLUMNS

↓

SORT DATA

↓

GROUP DATA

↓

CALCULATE SUMMARY

↓

GENERATE REPORT

Remember: dplyr makes data manipulation easier, faster and more readable in R.